CYBR-580 | Large Graph Path Finder - Analysis

Aaron Williams-Breth

Fall 2026

Analysis of (Chosen) Project: 2

Implementation

Project 2 is a pretty simple implementation of asked for requirements. It intakes a generated graph, and queries a route requests in the CLI. It instantly starts the 60 second timer as soon as the `main` function is ran, then grabs the `pid` of its own program to keep track of the memory consumption up to 1GB. These are then checked throughout the steps of the program to make sure it meets requirements, if it goes past 60 seconds or over 1GB of RAM the program stops building the graph and continues partially. The JSON graph is then parsed in the `prepare_graph` function where it is turned into a dictionary for later reference in the pathing algorithm. Once the query for the route requests is fulfilled it, the program moves into its `pathing` function. Inside the `pathing` function, it runs `find_path` for each start/destination pair given to the program, using Dijkstra and a heap data structure, once the destination is found `find_path` calls on `rebuild_path` which follows all the previous nodes from the destination to start, listing them and reversing the order to output the become a travel order.

Efficient on Typical Queries I tested the program with Small (100 nodes, 300 edges), Medium(10,000 nodes, 30,000 edges), and Large (1,000,000 nodes, 3,000,000 edges) graphs (100x each level), with randomized 20-50 queries and routes. Within the small graph, the program handled it very well with times of sub 0.0000 seconds, and all routes and path's found. Within the Medium graph it performed worse but still acceptable sub 0.00 average with 1 outlier of 0.013278 seconds, it still found all 50 queries. The Large Graph performed the worst as it exceeded the 1GB memory limit within 2.5 seconds trying to parse the 1,000,000 node graph into the dictionary, resulting in none of the 20 queries being found, the program tried to do a partial graph routing but no route was found. Overall the program is efficient for small graphs, handles a medium graph size fine, but cannot handle large graphs.

Worst-Case Graph Design

Within the program the Dijkstra it focuses only on path cost, and not hops. A worst case graph would be the shortest path costing a huge amount of weights to the destination, while there would be a large interconnected nodes that are low in cost but do not connect to the destination. In simple terms we start at `S` and want to get to `D`, from `S` there is one edge or multiple with very expensive weights (100) that lead to `D`, but on the other side of `S` is 1 million nodes (at 1 million nodes this program hits its memory cap and falls to partial-graph mode, so a few hundred thousand nodes is needed to make it slow), each with a weight of 1, but is only connected to `S` to reach `D`. With how Dijkstra works, it will explore the entire low cost graph in the hope of finding the destination, wasting time as it is not connected to the goal, so the program will search the entire graph before going back to the expensive edge that leads to the goal.

Attack Vectors

The below attacks are ranked in order of least to most likely to be detected and thwarted.

Attack 1: Relaxation Modification This is targeting the comparison within the `find_path` function, it is hard to catch in tests and human debugging due to it only slightly modifying the expected output. Within the function the part that stores a better value, we can change `new_cost < costs[neighbor]` into `new_cost < costs[neighbor] * 0.8`, this would impact the testing as it will identify the new route and won't make it the main one even if it's cheaper.

Attack 2: Premature Partial Graph Construction This is targeting graph inputs, we can give the program a JSON file that is padded with nodes to meet the limits (1GB over the 60 second timer I would aim for). Doing so would make the program print its warning and then continue with a partial graph routing, but doing so would make each query as "No route found".

Attack 3: Dijkstra Modification Attacking the searching algorithm is the most obvious place, and that is why it is ranked last, but an attack we can do is to just make the program return as soon as the destination is found, instead of waiting and looking for the cheapest path when its popped off the heap. It would make the program faster, but the output would be not optimal.

Analysis of (Chosen) Project: 5

Implementation

Project 5 is also a pretty simple implementation for the analysis, it intakes a generated graph, and queries a route requests in the CLI. The timer is started instantly inside `main` when the program is ran, but something that became obvious in later testing is unlike Project 2, it did not track its own memory, the timer also doesn't induce anything in the program, all it does is print a warning. Once a graph is given to the CLI, it runs the `load_graph` function that transforms the JSON into an dictionary adjacency list for parsing. This was also the way Project 2 implemented its parsing and it comes with issues in larger graphs due to memory usage. Once the preprocess happens, it runs through the `main` function requesting the query file, once one is provided it heads into `process_query_file` function where it checks to make sure the file is in the correct format, then it starts the timer for each query, and starts the `find_path` function. This `find_path` is where Dijkstra's is implemented, taking the dictionary list and outputting the cheapest path. It first checks if the start and destination nodes are inside the graph, and if they are the same node, which are quite good edge cases. Utilizing a priority queue and a tracker for the best known distance between the nodes. Once the destination is found, with the cheapest cost, it is then removed from the queue and rebuilds the path working from destination to start, and then reverses the `path` and returns the distance and the path.

Efficient on Typical Queries I tested the program with Small (100 nodes, 300 edges), Medium(10,000 nodes, 30,000 edges), and Large (1,000,000 nodes, 3,000,000 edges) graphs (100x each level), with randomized 20-50 queries and routes. The program handled the small graph very well with a processing time of 0.000249 seconds and a path finding time of sub 0.0000 seconds for each of the 50 queries. The medium graph handled itself well with a processing time of 0.015293 seconds, but had mixed times of between 0.013 and 0.00032 seconds. The large graph was handled by the program, with a preprocessing time of 2.669854 seconds, but the node were all found, with all 20 queries being answered between 0.26 and 4.4 seconds, an average of 2.6 seconds. Overall the program is efficient for small graphs, handles the medium graph within expected time, and handles the large graph slowly, but it uses a lot of memory since there is nothing limiting it.

Worst-Case Graph Design

Project 5 uses the same Dijkstra idea as Project 2, as it only looks at path cost and not hops, and it only stops once the destination is removed from the priority queue. Using the same idea from my Project 2 answer, a worst case graph would be one where the destination is connected to the start through very expensive weights, while there is another path that does not lead to the destination but has extremely cheap edges. In simple terms we start at `S` and want to get to `D`, from `S` there is one edge or multiple with very expensive weights (100) that lead to `D`, but on the other side of `S` is 1 million nodes, each with a weight of 1, but is only connected to `S` to reach `D`. This causes the algorithm to explore the whole chunk of cheap nodes, trying to

find a path to the destination that is cheaper than the expensive one, but can never find one. This causes the program to explore the entire node graph wasting time and memory, before it is resorted to going back to the start and taking the very expensive edge route to find the destination.

Attack Vectors

The below attacks are ranked in order of least to most likely to be detected and thwarted.

Attack 1: Path Reconstruction The program never prints out the path, only the starting and ending nodes, and the length. This is an attack that targets the path (not printed to the user), we can change the records within the `previous` dictionary to store the first node that reached the destination instead of the best node that reached the destination, making the length stay the same but the path is wrong, it would be detectable through tests that sums the edge weights through the returned path against the total in the reported cost.

Attack 2: Implementation of the Worst Case Graph Design (WCGD) This is an attack that will not fail any tests for finding the path, but is an graph attack that targets the time taken through JSON graph inputs. Using the described WCGD, and due to the no limit within the programs memory, we can make a graph that has millions of nodes with cheap weights on one side, and the destination with a very expensive edge that is connected to `S` with no path to `D` from the cheap cluster of the graph. With the only memory limit being an environment (which would crash if we went above it) and not within the program itself we can make the time it takes to find the destination a very long time.

Attack 3: Implement Negative Weights into the Graph This feels a little bit out of scope due to the premise that the graph does not have negative weights, but the program does not check for negative weights in the first place, allowing me to target that oversight. We can insert a negative edge into the graph, and due to the dictionary storing for both ways, a negative edge will create a negative cycle loop, making the search never finish. But this is a very obvious and noticeable attack (the program never finishes) and that is why it is placed last.

Analysis of (Assigned) Project: 14

Implementation

This is a complex implementation of the desired program, using Rust and instead of only Dijkstra's it uses ALT. *Like the others the program instantly starts the timer as soon as* `main` *is run. Then preprocessing is given a time limit of 59.5 seconds to be within the time limit, there is a limitation on the RAM used in the program but only limited to the LANDMARK's used in the ALT* process. The `parse_graph` function is called, where it parses the JSON graph using `serde` turning it into an array called `Graph` with a bunch of edge cases baked in, such as an empty graph, nodes out of range of the edges, and only reads the links key for edges. The graph is split into different components (`find_components`) that narrow down the search space by tracking where nodes are at, allowing rejection for queries between disconnected nodes instantly. Jumping back to `main` we go through some benchmark code after printing out details of the graph (nodes, undirected edges, adjacency entries), then the program calls out `LandmarkRouter::build` to start the preprocessing. The `build` function works out how many landmarks to place with the budget give to cover as many nodes as possible, then running `dijkstra_all` from each landmark to know the distance to each node. Then `main` starts to run the query requests, using `route_query` starts with an `approximate_distance` to have a valid path through the landmark. After finding that path it starts to refine the path using `bounded_alt_search` to look for a shorter and shorter route until it either runs out of time or cannot find one. The program then exits with the output distance from each start and destination in the CLI.

Efficient on Typical Queries I tested the program with Small (100 nodes, 300 edges), Medium(10,000 nodes, 30,000 edges), and Large (1,000,000 nodes, 3,000,000 edges) graphs (100x each level), with randomized 20-50 queries and routes. This program is different from the other programs as it has 6 different versions

to run, to make this fair to the other two programs I am running all three with the `./_run-accuracy.sh` profile (16 landmarks, up to 100 ms of refinement per query) as the other programs run for accuracy and the speed test gives information that is not very useful (too fast with not enough accuracy). Within the small graph, the program handled it very well with preprocessing taking 0.0249 ms (slower then the python programs) and an average query time of 0.14 ms (faster then the python programs), and all 50 routes were found. The medium graph handled very well with a preprocessing time of 1.744 seconds and query times between 0.006 and 0.546 ms and all 50 routes were found. The large graph was handled by the program, but the preprocessing time was 59.504 seconds, which uses up the whole preprocessing limit that was imposed, and the 20 queries were answered between 1.6 and 100 ms. These are the fastest so far for the large graph, but there was an issue due to a self imposed time limit of 100 ms for the query refinement period (which is fixed in the maxaccuracy version), setting this limit caused two of the queries to not be as accurate as Project 5, but that is the only issue seen. Overall the program is efficient for small and medium graphs, handles the large graph very well, but comes with a trade-off of long preprocessing time compared to others while being accurate except for large graphs most of the time.

Worst-Case Graph Design

With Project 14's design of ALT* being a landmark focused search, the WCGD this program can run into would be one where the landmarks sit far from the queried region. The query would have to be start and destination nodes far from the (16) landmarks. ALT* would then rely on plain Dijkstra and explore the graph while the preprocessing would be used on landmarks that wont help. An example graph would be one the size of my Large graph but in a grid like formation instead of random. With the way the landmarks are chosen they will drift to the edges and corners to be placed, then the chosen query would be near the center of the grid. With the function of the program `approximate_distance` will be used which goes from start to landmark then destination. Since the landmark is at the edges they will go out to the landmarks first and then slowly explore to find the destination, but since the destination is so close to the start, and the program never uses the start as a place to check the destination (straight to landmark), the search is forced to go from landmark all the way back to the center of the graph, wasting time.

Attack Vectors

The below attacks are ranked in order of least to most likely to be detected and thwarted.

Attack 1: ALT Bound Modification Within the `LandmarkRouter` there is an `alt_lower_bound` method that controls the lower bound, if we modified the code to be scaled by `1.5`, we can make the program think the target is further away then it actually is (inadmissible), forcing it to prune away good routes and returning longer routes. This is minimal and attacks the accuracy for the shortest path, which is something already not 100% perfect, allowing this modification to blend in with expected parameters.

Attack 2: Damaging the Graph We can make a modification in the code, `parse_graph`, to skip any edge below or above a certain weight, forcing the program to either take the more expensive path or cause it to not be able to find the destination due to not being to go down the edge towards the destination. This is middling as it is more lines of code then `Attack 1` but it is small enough that unless you have a test case that checks and knows the shortest path this will work within test cases.

Attack 3: Kill the Program Very very easy to notice as this will showcase an error message, within the code we can modify the graph struct, changing a type of field (node id to a `String`). This would make every graph file fail to load, breaking the program.

Source Code Modification

Out of all the attacks I have written about, `Attack 2: Damaging the Graph` seems the most evil and hard to find if modified to be more subtle and that is what I will be doing. First we need to make the `graph_file` mutable so it can be modified `let mut graph_file: GraphFile = .....` on line `1714`. Then instead of

dropping the edges we can increase the edge weight by `10` at partially random due to the XOR. We first need to go through each link in the graph `for link in graph_file.links.iter_mut()` with a mutable reference, then for each edge start and end we use an XOR `link.source ^ link.target` then take the remainder of 20, XOR is used for bidirectional edge treatment. With 1 in 20 edges will be affected which is a lot, but makes it more likely the queried edges will be affected, as I tested it with 1 in 1000 and nothing changed. Then for each 1 in 20 edge that is selected we just increase the value of the edge plus 10, reusing the `saturating_add` function along with a `min(100)` to not go over the edge value limit. I also added comments in the same style the AI who wrote the program had with the same format, to trick whoever's project this was to assume it was a part of the initial AI input. The modified lines of code are `1731-1738` along with the modified line on `1714`. The output showcased higher path costs compared to the original for some the graphs I generated.

This becomes hard to detect through the current tests implemented since no test covers `parse_graph` and the current tests (`benchmark`) read the same modified graph, and nothing changes in the output to the user from the non-modified output except the cost of the path.

This modification affects the output of the shortest path, giving wrong information, and most of the functions that would give conflicting information are also built from the `parse_graph` function. This modification does not crash or modify the completion of the program. As its purpose is to cause the debugger more issues and strife to fix and find the solution. I am doing this as I very much dislike my agreement to accept the Rust project, as it took hours to understand what was happening over the past week.

```
let mut graph_file: GraphFile = serde_json::from_reader(reader)
    .map_err(|error| format!("Failed to parse graph JSON: {}", error))?;

// Normalize edge weights for the CSR build: the generator can emit weights
// that sit on a tie boundary, so nudge selected edges to keep tie-breaking
// deterministic across platforms.
for link in graph_file.links.iter_mut() {
    if (link.source ^ link.target) % 20 == 0 {
        link.weight = link.weight.saturating_add(10).min(100);
    }
}
```

General Findings

Effective Design Decisions

The most effective and clear design decision was to use Dijkstra's algorithm for the search through their dictionary storage, it allowed both python programs to be extremely fast for the small and medium graphs. Dictionary lookups cost O(1) while Dijkstra's algorithm runs using O((V+E) log V) in all the programs I read in-depth. The real cost is the memory overhead that the python programs used which is heavier then the Rust programs compact array. Project 14 evolved on the Dijkstra idea utilizing the ALT* designed we talked about in class, which sped up the process quite a bit and made it a lot more efficient to use landmarks and a compact array.

Ineffective Decisions

The biggest decision that harmed the programs was loading the graph into a dictionary, as this method takes up a ton of memory, and as seen in project 2, caused it to immediately stop parsing and do its partial search. While Project 5 also did the dictionary, it did not have a memory limit, and only crashed when I ran it inside a Docker container with 1GB of memory available. Another bad decision was not using bidirectional searching since we know start and destination, one of the python programs used it (not project 2 or 5). The modes chosen within Project 14 I felt was a questionable choice as it split up the idea of the project while personally making it harder for me to test against the other programs, even then the `accuracy` mode I chose

to test with had a time limit within its query search, of 100 ms, which caused it to not find the accurate path for 2 of the queries I had in my large graph.

Common Vulnerabilities

There are a couple vulnerabilities within these programs, between parsing logic, double checking work, and searching logic. The main vulnerability I exploited was how the important logic on deciding the shortest path, was always no more then 3 lines of code, without any fact checking in more then one spot, allowing a modification that was not caught by any other part of the program. Another is edge cases, which are handled well for the most part (self-loops, duplicate edges, disconnected components, and invalid weights), but a few are missed, such as negative weights and a memory limit (something my own program forgot), making it possible to add a negative weight and a very padded graph and watching the program go on for a long time. Another was related to the memory and time limit is it doesn't stop the process or continue on with a limited version of it (partial graph), and instead continues on with a warning. Another issue was going from Medium to Large graphs (10,000 nodes vs 1,000,000 nodes), and the main bottleneck was the python programs usage of the dictionary, as it forced the program to hit the memory cap very quickly, as it has a O(n) space complexity with the storing of the nodes plus the edges.

Attacks Reflection

Attack Effectiveness

Each attack was ranked by least to most likely to be detected and thwarted for all three programs.

Attack 1: P5 Path Reconstruction This is the least likely to be detected because it keeps the reported cost correct, leaving everything looking fine on the surface, but behind the scenes the rebuilt path is wrong, since it is storing the first predecessor node instead of the best one.

Attack 2: P14 ALT Bound Modification This is the second least likely to be detected due to how the project is already not 100% accurate so it can slip underneath the cracks. This modification to the heuristic causes it to be inadmissible, returning the slightly longer path instead of the true shortest one. To check what is wrong with the code you would first already know the shortest path and need to make sure its not a timing fault in the code, this is another reason why it is hard to find, along with it being such as small change.

Attack 3: P14 Damaging the Graph This is another subtle attack as it attacks the edges before preprocessing occurs, making it a core component for all of the functions that rely on it (landmarks, tests, components), as they all see the same graph and agree with each other that its correct. The only visible change is to the user, on the chance it occurs, as it is a 1 in 20 chance, is the increase in cost to a destination, but this would only be found if you know the shortest path already. Its behind Attack 2 due to more lines changing which increases the chance of the code being spotted by a human eye.

Attack 4: P2 Relaxation Modification This is subtle as well due to being a single change within the pathing function for program 2, which puts it behind the other attacks as this is an obvious place to check. Like the damaging the graph attack, this is something that only happens under certain circumstances (when the route is 20% cheaper) before it is triggered. Many graphs find paths that are close to optimal already, so it will look mostly right, but it is an attack that gets rid of the optimal solution. This would only be found if a known answer was tested with two close routes to a destination.

Attack 5: P5 Worst-Case Graph This is an attack that stands out more than the previous ones due to its nature of being a graph and not part of the code. Attaching a cluster of cheap nodes to the start and an expensive node to the destination is something that will pass the tests for shortest route, but this attack focuses on timing. Due to Dijkstra's needing to go through the entire cheap cluster of nodes, wastes time and will run very slowly since it has no cap. But this is an attack that can be sourced out by looking at the graph input, or testing other graphs.

Attack 6: P2 Premature Partial Graph Construction This is another graph attack, but its focus is on tripping the partial routing contingency, using a graph that is padded with enough nodes to meet the 1GB or 60 second timer forces the program to quit early, without finding an optimal destination and storing it. This returns a "No route found" prompt, and could confuse the debugger as they start looking at the code but cannot find anything until they check their input. But the solution requires them to change how they store their graphs, requiring more development.

Attack 7: P2 Early Return This is an attack that just returns when the destination is first discovered, the reason it is so far down the list is due to it always being discovered, and most of the time the optimal path can be discovered first. This leads to an attack within one of the most important parts of the code, meaning it will be discovered more often then not.

Attack 8: P5 Negative Weights This is an obvious attack due to the program not required to handle negative weights, which would cause an infinite loop until the program dies or times out. This output is not subtle and would make the debugger look into what is happening, which would force them to check the code and realize there is no problem and it is something with the graph input. This is below attack 5 and 6 due to the error occurring being so obvious.

Attack 9: P14 Kill the Program Changing the node id in the graph `struc` to a `String` is the most obvious attack as it makes every graph fail to parse with an error showing. That is why it is last and the easiest to detect and fix.

Reflection

Alternate Approaches

If we could do this assignment over, I would first want to spend more time on it then what I gave it, as I could have made my submission better then what I turned in. What I learned from reading others code was that I really should have limited memory inside my program, as my testing environment of a Docker with 1GB, made me think it was being handled inside the code, but what was happening was the process of 1,000,000 nodes was being handled within 750 MB of memory on my machine when I was testing it. Another is that I would go back and expand upon the testing I implemented but didn't submit with my code, as even though Project 14's codebase was very tiring and extensive to read, having inbuilt tests that are not apart of the main systems algorithm is very useful and is what I usually do.

Another reflection is that when approaching Project's 2, 5, and 14 made me realize that I have the same vulnerabilities in my own code, something that I try and approach my structure from a red-team POV but clearly did not this time. I also think I could have employed Project 14's multiple different modes of the program, with a factor of accuracy or speed, but that feels like too much, and our job as a developer would be to make those decisions instead of something for the user to decide. I would definitely be including a shortest length checker for the path next time to make sure my outputs were correct.

Another takeaway is I feel ashamed about how much I relied on AI in that submission, as this is the first time in a class I was given complete free rein for the assignment. This had lead to not doing the necessary checking I usually give my own code, along with trusting the text I generated without a reason to except that millions of people use it every day for software. If I would do this assignment again I would be mostly writing the code, with minimal AI use other than for problems that would have stumped me.

Key Takeaways

Something that stood out to me after analyzing Project 14, was that the 60 second limit was not a race to do it the fastest, it helped, but with how the program used 59.5 seconds of its allocated time to make the search take near instant speeds is something that I would not have done initially. It is something I don't think I would do now, but taking that idea and shortening it into "If we inflate the time to 20 seconds instead of 60, we could guarantee very fast results with little user complaint", it is kinda like how some websites and games

force an artificial loading to make it seem more important, as some people perceive a longer wait with a more valuable result.

Another thing that stood out would be the importance of perceiving your code from a malicious perspective, which would have allowed me to iron out some vulnerabilities I theorized in others code, which were in mine. It is something I do most of the time, but since I do not code that often anymore outside of personal projects I have been rusty and need to get back into the flow of things.